NORTHEASTERN MINDANAO STATE UNIVERSITY
TANDAG CAMPUS

AI Engineering Portfolio

Student Information Management System

School Year 2026-2027

Student name	Ace R. Cabuga
Instructor	Angel Abellanosa, Jr.
Course/Section	PHP Web Development Framework: Laravel
Submission date	

DRAFT - STUDENT REVIEW REQUIRED

Status: DRAFT. Student identity, original prompt review, personal evaluations, and live application screenshots are incomplete. Do not present this export as a final portfolio.

Current evidence snapshot - November 10, 2026

The repository has 46 commits and 20 screenshot files. A fresh local test run passed 93 tests with 633 assertions and no failures or skips. The current verification record also documents 61 routes, 13 applied migrations, passing SQLite integrity/foreign-key checks, a passing production build, Composer validation, and Pint. See for details.

The official handbook and rubric PDFs were not in the workspace, so their full contents could not be checked. The screenshots are authentic development evidence, but none depicts a running SIMS page or an API-client exchange. Personal evaluations and missing original prompts remain student-owned.

Integrity note: The agent architecture is a proposal. Repository evidence does not establish independent specialist-agent execution, model comparisons, or token measurements.

1. Introduction

SIMS is an existing Laravel 12 student information system for NEMSU Tandag Campus. Its documented features include Breeze authentication, Administrator and Staff permissions, Student/Subject/Enrollment CRUD, a Sanctum Student API, and a College/Department/Program catalog. The application uses Blade, Tailwind CSS, Alpine.js, Eloquent, SQLite, PHPUnit, and Vite. AI assistance in the available project history was used for focused implementation, debugging, testing, code review, and documentation; a generated suggestion is not proof that the student used or accepted it.

This draft records prompts and outcomes present in the project notes and available conversation history. A summary is not a verbatim prompt. Verify every entry against personal chat history before submission.

2. AI Tools Used

1. Tool visible in this current workflow: GitHub Copilot in VS Code.
1. Model/version: not exposed in the available evidence.
1. Purpose: inspect the existing Laravel workspace, make code changes, execute application commands/tests, and update project documentation.
1. Observed strengths in this workflow: workspace file inspection, focused patches, route/test execution, and iterative corrections in one environment.
1. Observed limitation: the session record does not identify a model version or provide token-cost telemetry.
1. Independent specialized agents: no independent handoffs are evidenced.
1. Historical screenshots show other AI interfaces, but open tabs alone do not establish that those tools were used. The student must identify any other tools from their own records.
1. Other AI platforms tested for comparison: none documented.
1. The PDF export exists in and is marked DRAFT.

3. Prompt Engineering Methodology

The recorded workflow includes short feature prompts, context-aware preservation constraints, follow-up debugging prompts based on actual errors, testing prompts that request real counts, and documentation prompts that require source verification. Refinements added concrete file/module scope, role permissions, non-destructive database constraints, expected outputs, and checks. Examples below are limited to excerpts retained in project history; missing original wording is labeled rather than reconstructed.

Case Study Index: Ten Cases

Each case below points to the available prompt/result record. Historical screenshots are evidence of what they visibly show, not proof of an unpictured application behavior. The student evaluation is intentionally blank in every case.

1. **PHP namespace debugging.** Original error excerpt and correction: Prompt 1. Relevant file: . Verification: historical test evidence; current full suite is separate. Screenshot: , a historical debugging capture. **Student evaluation:** [STUDENT INPUT REQUIRED].
2. **Breeze navigation.** Original excerpts: Prompt 2. Result: desktop and responsive management links. Relevant file: . Verification: project history and current route availability; no live navigation screenshot. **Student evaluation:** [STUDENT INPUT REQUIRED].
3. **Preserving the existing project.** Original excerpt: Prompt 3. Result: retain Laravel/Breeze, existing modules and data; avoid destructive resets. Relevant files: existing routes, migrations, and feature tests. Verification: current application and migration checks. No dedicated screenshot. **Student evaluation:** [STUDENT INPUT REQUIRED].
4. **Student REST API.** Prompt excerpts: Prompt 4. Result: five Sanctum-protected Student operations and role-specific access. Relevant files: , , , . Verification: current route list and passing API tests. Screenshot: , historical route/middleware output. **Student evaluation:** [STUDENT INPUT REQUIRED].
5. **API route-name conflict.** The original correction prompt was not separately preserved; the problem and fix are documented under Prompt 4. API route names received the prefix to protect web route generation. Relevant files: , , . Verification: 61 current routes and passing suite. Screenshot: same historical route capture as Case 4. **Student evaluation:** [STUDENT INPUT REQUIRED].
6. **Invalid Laravel validation rules.** The original implementation prompt is unavailable; the later testing instruction is quoted in Prompt 5. The invalid entry was split into Laravel rules . Relevant files: and API tests. Verification: current suite passes. No screenshot directly shows this correction. **Student evaluation:** [STUDENT INPUT REQUIRED].
7. **Incorrect Student factory assumption.** No verbatim original prompt was preserved. A test assumption about was replaced with explicit model creation. Relevant files: Student API and CRUD feature tests. Verification: current suite passes. No screenshot directly shows the correction. **Student evaluation:** [STUDENT INPUT REQUIRED].
8. **NEMSU academic catalog.** The objective and constraints are preserved in Prompt 6, but the original prompt text is unavailable. The College/Department/Program catalog remains separate from Subjects and legacy data. Relevant files: catalog models, migrations, seeder, controller, and . Verification: current database contains 7 colleges, 13 departments, and 22 programs; catalog tests pass. No screenshot directly shows the catalog UI. **Student evaluation:** [STUDENT INPUT REQUIRED].
9. **Administration Dashboard.** Prompt evidence and requirements are summarized in Prompt 7; verify wording against the student's own history. Result: data-backed dashboard and protected account management. Relevant files: , , and administration feature tests. Verification: current suite passes. No live dashboard screenshot exists. **Student evaluation:** [STUDENT INPUT REQUIRED].
10. **Security and testing corrections.** Prompt excerpts in Prompt 7 require Administrator-only role changes and last-Administrator protection. Result: server-side checks and regression tests; API writes are also Administrator-only. Relevant files: , , , and related feature tests. Verification: fresh full suite passes 93 tests / 633 assertions. Screenshot: is historical partial test output, not this fresh run. **Student evaluation:** [STUDENT INPUT REQUIRED].

Original text is unavailable for Cases 5-8 where noted. Do not rewrite a summary as if it were the verbatim prompt. For every case, the student should add their own evaluation and any genuine chat capture before final submission.

4. Actual Prompt Case Studies

Prompt 1 — debugging PHP namespace placement

Objective: Resolve a test-file fatal error preventing Laravel tests from starting.

Original prompt excerpt (user):

> Fatal error: Namespace declaration statement has to be the very first statement or after any declare call in the script in C:\laragon\www\student-information-system\tests\Feature\SubjectCrudTest.php on line 4

Problem identified: PHP does not allow bytes or output before the opening PHP tag when a namespace declaration follows.

AI result / correction recorded: The leading blank line before was removed in the affected test file. The same issue was checked in other touched PHP files.

Verification evidence: The Subject CRUD test later passed, and route loading was verified during local-run troubleshooting. Preserve the actual terminal screenshot or test output from the student's environment if it is required as submission evidence.

Student evaluation: Add a first-person assessment of whether this fix addressed the reported failure and what was checked before changing the file.

Prompt 2 — adding management navigation

Objective: Make the existing management module accessible from the Breeze navigation.

Original prompt excerpts (user):

> move the manage students to nav bar

AI result recorded: Management links were added to Breeze desktop and responsive navigation, preserving route-based active states.

Verification evidence: Navigation changes were made in the existing Breeze layout; prior project work reports Blade and related feature checks. Add screenshots showing desktop and mobile navigation if required.

Student evaluation: Describe whether the placement and responsive behavior matched the intended navigation and any manual browser checks.

Prompt 3 — refining the scope to the existing project

Original prompt excerpt (user):

> This is an existing project, NOT a new project.

> DO NOT recreate Laravel, overwrite working modules unnecessarily, replace Breeze authentication, reset the database, or destroy existing data.

Refinement: The prompt then supplied detailed Laravel, authorization, CRUD, database, API, test, and documentation requirements and directed the work to inspect before implementing. This bounded changes to the existing architecture and data.

Evaluation: Existing routes, migrations, role rules, and tests were inspected before adding the Student API. No new Laravel application was created, and only the additive Sanctum token migration was applied.

Prompt 4 — implementing and validating the required Student API

Objective: Implement all five required REST endpoints without replacing existing authentication or schema.

Original prompt excerpts (user):

> THIS IS A HIGH-PRIORITY REQUIREMENT.

> Provide tests for all five mandatory endpoints.

> Use the existing Student database schema.

AI result: Added Sanctum bearer-token authentication, five Student API routes, an API controller and resource, shared student validation, and admin-only API writes. Kept the web CRUD controller and database schema.

Problem found during validation: The API resource initially reused the web route names. Route inspection and a web CRUD test exposed that resolved to the API route. API route names were prefixed so web route generation remains unchanged. Tests also showed there is no Student factory; test data was changed to explicit model creation rather than adding an unnecessary factory.

Verification result at that development stage: , , and passed in the then-current full run (56 tests / 280 assertions). Later catalog work expanded the test suite; see Prompt 6 for final verification totals.

Student evaluation: Review the authorization policy, payload validation, and response codes; document any changes you personally made or rejected.

Prompt 5 — validation prompt and iterative failure correction

Actual validation prompt excerpt (user):

> Run:
>
> Report:
>
> 1. Tests executed
> 1. Tests passed
> 1. Tests failed
> 1. Actual error messages
> 1. Fixes applied
>
> Do not claim that tests passed unless they actually ran successfully.

AI-generated output: Added a partial-update test for the Student API and used as one string inside an array-based Laravel rule set.

Problem found: The full test run reported:

The partial-update request returned HTTP 500 instead of the expected 200.

Correction applied: Changed the partial required rule to the two Laravel rule entries , while preserving the existing full PUT rules.

Result validated at that stage: The targeted Student API and CRUD tests passed after the correction. The complete suite then passed with 56 tests and 280 assertions; this historical count was superseded by the latest result below.

A separate subsequent test run briefly reported because an email-uniqueness assertion was placed in the student-update test instead of the student-create test. The assertion was moved into the test scope where is defined; the targeted CRUD suites and final full suite then passed.

The schema-export smoke test also caught an initial DDL ordering problem (while importing an index before its table). The export query was changed to emit table definitions before indexes and excluded SQLite's internal ; the resulting schema-only SQL successfully imported into an in-memory SQLite database.

Prompt 6 — integrating the supplied academic catalog

Objective (carried-forward task summary; verbatim original prompt is not available here): Add the supplied NEMSU Tandag Campus 2026–2027 college, department, and program hierarchy to the existing SIMS without creating subjects from degree programs, changing the existing enrollment relationship, or replacing legacy student course data.

Implementation result: Added normalized College → Department → Program tables/models and an idempotent, non-destructive seeder; connected dependent student form selectors and server-side relationship validation; retained the Student string and Student → Enrollment → Subject design; added an authenticated searchable directory and dashboard catalog counts.

Problem identified during review: Simply making a catalog-only choice would reject older student values on update. Passing selector IDs

through mass-assignment was also unnecessary because only the selected program name belongs in the existing Student schema.

Correction applied: Existing unchanged legacy course values remain valid on updates; web and API controllers persist only the Student's existing fillable fields. The seeder does not overwrite a college if an abbreviation conflict is found; it reports the conflict rather than silently changing catalog data.

Verification result: The focused catalog, Student CRUD, Student API, and dashboard tests passed (16 tests, 149 assertions). The full suite then passed 62 tests / 324 assertions. The local additive migrations and seeder completed; verification showed 7 colleges, 13 departments, 22 programs, and unchanged counts of one student, one subject, and one enrollment. The production frontend build, Pint on changed PHP files, Composer validation, and schema-only export import also passed. The schema import contained no catalog rows.

Student evaluation: Check the supplied catalog against its original source and describe whether keeping a string-valued Student field is an appropriate compatibility choice for the final project.

Prompt 7 — upgrading the administration dashboard and role management

Original prompt evidence: The user supplied a detailed Administration upgrade brief in this conversation. Its explicit requirements included:

> Transform it into a useful system management dashboard.

> Only Administrators can change account roles.

> Prevent: Unauthorized privilege escalation; Users changing their own role through profile requests; Accidental self-demotion by an Administrator; Demoting the last remaining Administrator.

> Never fabricate historical activity.

Inspection findings: The prior Administration page was a static welcome view. already required and was linked by the existing dashboard and navigation. Registration already assigned , and accepted only name/email while excluded role. Breeze profile/account flows, Student/Subject/Enrollment CRUD, the API, and the academic catalog were preserved.

Implementation result: Replaced the static page with an invokable admin dashboard controller and live totals; added summaries, management shortcuts, catalog/program details, recent records, and data-backed program/year/status reports. Added an administrator-only searchable and filterable user directory, safe account detail pages, and role-only updates. No admin account deletion, suspension, or historical audit log was added. Recent records are labeled by their creation timestamp, not presented as action history.

Security review and correction: The user-edit screen is not the security boundary: the role update is restricted by authenticated admin middleware and server-side validation. A role-only endpoint avoids mass-assigning user attributes; self-role changes are rejected, and the update is transactional with a post-change check that rolls back any change leaving no administrator. The existing profile update already ignored a crafted field, and a

regression test now verifies this.

Verification evidence at that stage: The new dashboard/user-management tests cover database totals, reports, catalog counts, route access for staff/guests, search/filtering, role validation, promotion/demotion of another account, self-demotion protection, and profile role tampering. The complete PHPUnit suite then passed **70 tests / 374 assertions**; passed; Pint passed for the changed admin PHP files. No migrations or destructive DB commands were run for the Administration changes.

Workflow note: This was completed in the current assistant session. No independent specialist agent handoffs or screenshots are claimed.

5. Prompt Iteration Comparison

Prompting change visible in the record	Why the refinement mattered	Verification boundary
Brief navigation request followed by desktop and responsive Breeze link context	Named the existing layout surfaces and expected route-based state	The screenshots show source/terminal work, not a live navigation browser check.
Existing-project constraint followed by explicit preservation and no-reset instructions	Kept work within the existing Laravel application and its records	Current tests and migration checks pass; no destructive migration command was run.
Student API requirements expanded to list all five endpoints, roles, validation, and tests	Made the API contract and authorization testable	Current route list and Student API feature tests pass.
Testing request required actual failures and counts instead of assumed success	Surfaced the invalid partial-update rule and kept historical counts distinct	Current full result is 93 passed / 633 assertions.

The exact first/revised wording is not available for every case. The table compares only prompt details preserved in the project records.

6. AI Mistakes and Corrections

Verified issue in project notes	Correction recorded	Check/evidence
PHP namespace file began with content before	Removed the leading blank line	Historical namespace-debugging screenshot; current test suite passes, but does not independently replay the old failure.
API resource reused web Student route names	Prefixed API route names with	Current 61-route listing and web/API tests.
Partial update used as one validator rule	Split it into	Current Student API tests pass.
A test assumed a nonexistent	Created explicit test records instead of adding an unnecessary factory	Current Student CRUD/API tests pass.
A test referenced outside its scope	Moved the assertion to the test that defines the record	Historical fix recorded in project notes; no separate screenshot.
Schema-only export put an index before its table	Exported table definitions before indexes and excluded SQLite internals	Historical schema-import test recorded in project notes.
User/API policy documentation and implementation diverged	Aligned API write middleware and regression tests with Administrator-only writes	Current route middleware and role tests.

These are project-history notes, not proof that the student personally authored or reviewed each correction. Add personal assessment only from firsthand experience.

7. Token and Context Management

The actual workflow used progressively broader context rather than copying the entire repository into each decision. Examples below describe actions visible in the development session; no token totals or model-specific token prices were recorded.

Technique	Concrete example from this work	Quality check
Single-file context	Inspected separately when reviewing the shared validation behavior.	Compared its rules against the student web/API tests and route binding.
Feature-based context	Read Student CRUD and Student API tests together while filling student validation, auth, and not-found gaps.	Ran the relevant test filter before the full suite.
Module-based context	Reviewed Subject, Enrollment, Dashboard, RoleAccess, and Breeze authentication tests as separate feature modules.	Added only module-relevant assertions and preserved existing role behavior.
Smaller task prompts	The actual requests “put save bbutton in enrollments” and “move the manage students to nav bar” addressed bounded UI changes; the current request separately specified validation, docs, and portfolio requirements.	Confirmed the corresponding Enrollment/navigation tests and later ran the full suite.
Task decomposition	Work proceeded through inspect, focused implementation, targeted tests, documentation, and full verification rather than a single broad code-generation step.	Each stage had observable tool output; the current test additions pass their targeted run.
Incremental generation	API implementation was followed by tests, then route naming was adjusted after a regression surfaced.	The route list and regression test confirmed web/API route names no longer collide.
Context reuse	A carried-forward conversation summary retained project constraints, prior decisions, and test results after earlier conversation turns were summarized.	The summary was checked against current files and commands before relying on it.
Agent specialization	Specialist responsibilities are documented as a proposed workflow.	No independent agents or specialist handoffs are claimed as actual work.

Context-management questions retained in earlier notes

The official handbook PDF was unavailable, so the wording below cannot be confirmed as verbatim handbook text. Compare each question with the official copy before citing it as a course requirement.

1. **How did unnecessary AI context get reduced?** By reading likely relevant files in bounded groups (for example, student request + Student tests), checking the specific migration and route list rather than re-reading the entire application, and using the carried-forward summary to retain prior decisions. This describes context organization, not a measured token reduction.
2. **How was prompt repetition avoided?** The summarized history was reused for already-set project constraints; subsequent work focused on the remaining tests and documentation. Actual earlier prompts are quoted above rather than reconstructed as new historical prompts.
3. **How were AI-generated outputs verified?** With route inspection, targeted feature tests, full PHPUnit runs, Pint, Composer validation, production asset build, migration status, and before/after record counts. The route-name collision, missing factory assumption, and formatting changes are recorded with their corrections below.
4. **How were token cost and output quality balanced?** Tests were first run for affected feature modules, then the complete suite was run; this gives fast feedback without treating narrow success as final proof. Existing controllers, schema, and middleware were reused rather than regenerated. No token cost telemetry was captured, so no numerical savings are claimed.

Critical verification examples

API route naming

1. **AI-assisted implementation:** Added a resource API for the same Student model and resource names used by the web UI.
1. **Problem found:** Laravel's shared route-name namespace caused to resolve to the API route, breaking the web test's redirect expectation.
1. **Correction applied:** Put API resource routes under the name prefix, preserving the existing web route names.
1. **Result validated:** shows and independently; the web and API tests pass.

Test fixture assumption

1. **AI-generated test setup:** Initially called even though this project has no Student factory.
1. **Problem found:** The test runner reported .
1. **Correction applied:** Used explicit Student records in tests instead of adding an unnecessary factory solely for these cases.
1. **Result validated:** Student CRUD and API tests pass using the actual project setup.

Formatting and migration safety

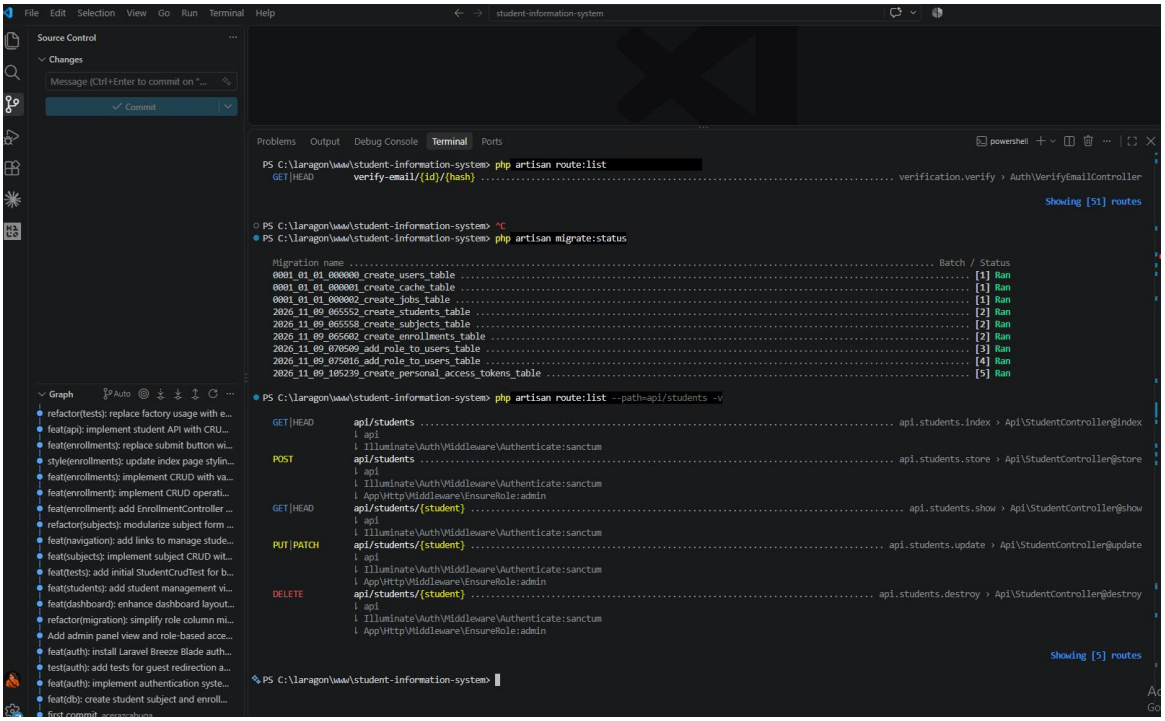
1. **AI-assisted changes:** Added API classes and a Sanctum personal-token migration.
1. **Problem found:** Pint reported formatting differences; the token migration was pending and needed to be checked before modifying the local DB.
1. **Correction applied:** Ran Pint on changed PHP files, reviewed the migration as additive, then applied it without a reset.
1. **Result validated:** Pint passed; migration status showed the migration applied; student, subject, and enrollment counts remained one each.

8. Verification Evidence

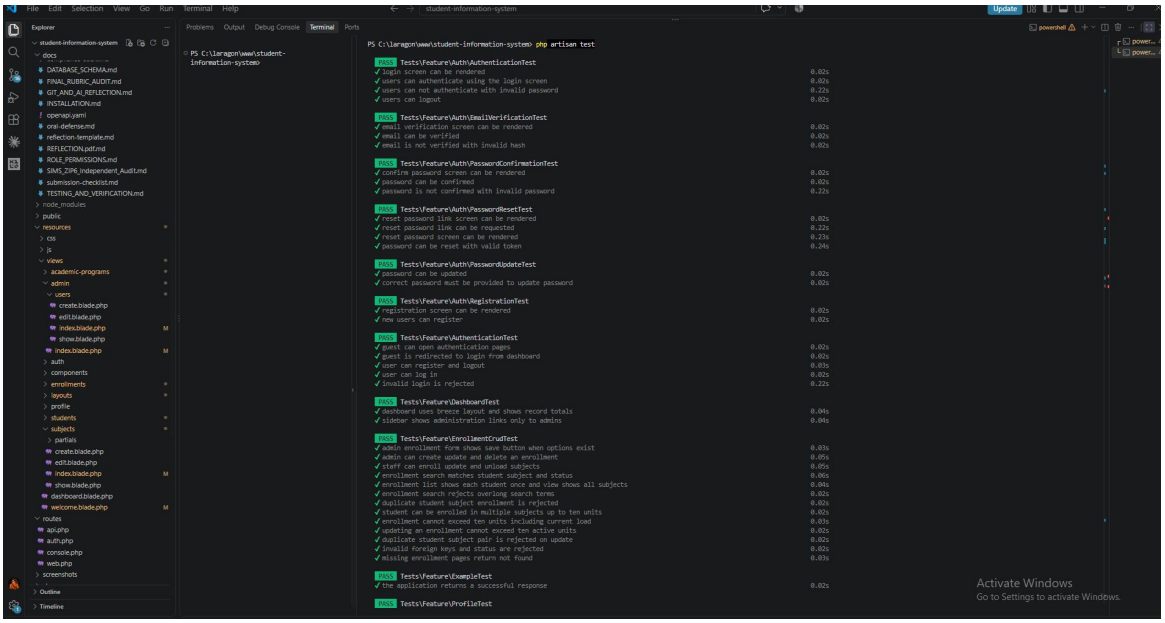
The fresh Windows/Laragon run passed **93 tests / 633 assertions**, with no failures or skips. The route list contains 61 routes, all 13 migrations are applied, SQLite integrity and foreign-key checks passed, Composer validation passed, Pint passed, and the Vite production build passed with a runtime image resolution warning. Full results and limitations are in .

Selected historical screenshot evidence

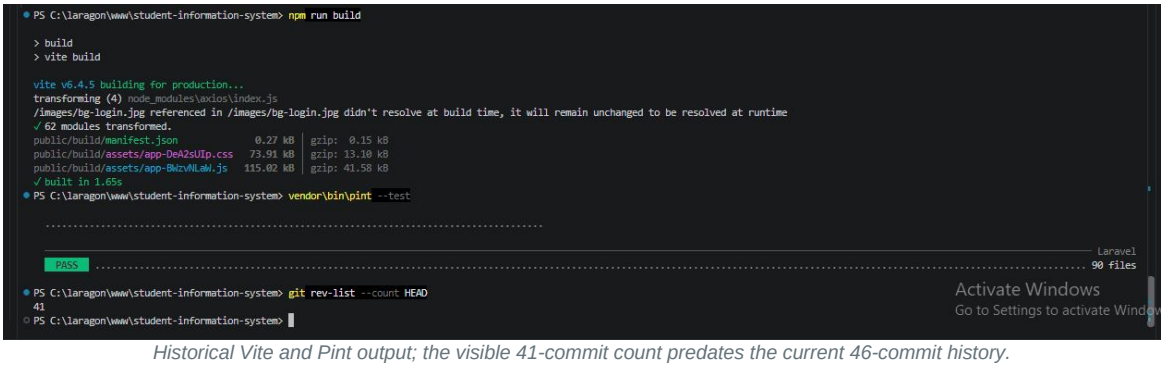
These are real screenshots from the collection. Their contents are historical and do not replace the fresh checks above.



Historical verbose Laravel API route and middleware listing; the screenshot predates the current route check.



Historical PHPUnit run in progress; the final summary is not visible in this capture.



Historical Vite and Pint output; the visible 41-commit count predates the current 46-commit history.

Evidence still to add

1. Screenshots of the running dashboard, Student/Subject/Enrollment pages, and API requests and responses; none are in the current screenshot collection.
1. Exact model/version metadata and complete original prompt captures from the student's chat history.
1. First-person evaluation for each case, plus the student's conclusion and confirmation of AI tools actually used.
1. Student review and completion of the DRAFT PDF cover-page fields.

9. Student Evaluation and Conclusion

Use the case-study prompts above to assess what was useful, what needed correction, what was personally verified, and what would be changed next time. Do not submit generated first-person opinions as the student's own.

[STUDENT INPUT REQUIRED] Write a conclusion in the student's own words, including a specific prompt that improved, one AI error caught by verification, and one lesson about responsible AI use.